Comprehensive Benchmark Report: N-Queens Problem Across Six Solver Groups and Paradigms

DatNK0116[a]

[a]*Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam*

ARTICLE INFO

Keywords:
N-Queens
Propositional Satisfiability
Constraint Programming
Mixed-Integer Programming
Benchmarking
PySAT

ABSTRACT

This report presents an exhaustive empirical evaluation of solving the classic N-Queens problem across six major solver groups and modeling paradigms: Propositional Satisfiability (SAT with CaDiCaL, Glucose3, MiniSat22, Kissat, and MapleCM), Constraint Programming via CP-SAT (OR-Tools), Mixed-Integer Programming (MIP with CBC, SCIP), High-Performance Commercial MIP (Gurobi), Commercial CPLEX MIP, and Commercial CPLEX CP. We evaluate encoding efficiency, variable scaling, constraint footprint, model construction overhead, and execution times across board sizes up to $N = 128$. Detailed performance metrics and visualizations highlight the trade-offs between expressive global constraints and low-level clausal representations, alongside comprehensive timeout analyses, CP solve overhead management, and solution status semantics (FEASIBLE vs OPTIMAL).

1. Introduction

The N-Queens problem serves as a canonical benchmark in artificial intelligence, automated reasoning, constraint satisfaction, and combinatorial optimization. Placing N non-attacking queens on an $N \times N$ chessboard tests the scalability, constraint propagation, and search pruning power of modern discrete solvers.

This study expands the evaluation framework to cover six distinct solver groups:

- **SAT Solvers**: CaDiCaL [1], Glucose3, MiniSat22, Kissat, and MapleCM evaluated across Pairwise, Binary, Sequential Counter, Commander, and Product cardinality encodings (implemented using the PySAT toolkit [4]).
- **Constraint Programming (CP-SAT)**: OR-Tools CP-SAT using Integer, Boolean, and Allowed assignments configurations [6].
- **Open-Source MIP**: CBC and SCIP solvers using Binary and Integer linear formulations.
- **Commercial MIP (Gurobi)**: High-performance Gurobi optimization engine with advanced presolve and branch-and-cut routines (noting trial license bounds at $N \leq 44$).
- **Commercial CPLEX MIP**: IBM ILOG CPLEX Optimizer using the Mathematical Programming (MIP) engine.
- **Commercial CPLEX CP**: IBM ILOG CPLEX Optimizer using the Constraint Programming (CP) engine, scaling up to $N = 128$.

2. Encodings and Mathematical Formulations

Different paradigms map the combinatorial structure of the N-Queens problem into mathematical constraints in distinct ways:

24021404@vnu.edu.vn (DatNK0116)
ORCID(s):

[1]Course: Modern Topics in Computer Science (2627I_INT3011E_4)

2.1. Propositional Satisfiability (SAT) Encodings

Boolean variables $x_{i,j}$ denote whether a queen is placed at row i, column j ($1 \le i, j \le N$).

- **At-Least-One (ALO)** per row: $\bigvee_{j=1}^{N} x_{i,j}$ for all i.
- **At-Most-One (AMO)** per row: Pairwise encoding $\bigwedge_{1 \le j < k \le N} (\neg x_{i,j} \vee \neg x_{i,k})$ yielding $\Theta(N^2)$ clauses; advanced encodings including Klieber & Kwon's Commander encoding [3] and Chen's Product encoding [5].
- **Binary and Sequential Counter Encodings**: Encodes cardinality constraints using $\lceil \log_2 n \rceil$ auxiliary boolean variables or Sinz's sequential counter networks [2].

2.2. Constraint Programming (CP-SAT) & Global Constraints

OR-Tools CP-SAT utilizes integer decision variables $q_i \in \{0, 1, \ldots, N-1\}$ representing the column of the queen in row i. Global constraints enforce:

1. AllDifferent$(q_1, \ldots, q_N)$ (columns)
2. AllDifferent$(q_1 + 1, q_2 + 2, \ldots, q_N + N)$ (main diagonals)
3. AllDifferent$(q_1 - 1, q_2 - 2, \ldots, q_N - N)$ (anti-diagonals)

This formulation utilizes N primary integer decision variables, alongside auxiliary variables introduced during model construction (resulting in 383 variables for $N = 128$ in the integer configuration proto), and 3 global constraints, pruning infeasible branches effectively.

2.3. Mixed-Integer Programming (MIP)

MIP models formulate the problem as linear inequalities over binary variables $x_{i,j} \in \{0, 1\}$ or integer variables $q_i \in \{0, 1, \ldots, N-1\}$, coupled with linear diagonal separation cuts:

$$\sum_{i=1}^{N} x_{i,j} = 1 \quad (\forall j), \quad \sum_{j=1}^{N} x_{i,j} = 1 \quad (\forall i), \quad \sum_{(i,j) \in \text{diag}} x_{i,j} \le 1$$

3. Experimental Setup

Benchmarks were executed on macOS Sequoia with Python 3.14, utilizing isolated subprocesses with a strict 30-second hard timeout applied to the entire child process (encompassing library import, model construction, and solving: `import_time + build_time + solve_time`). Board sizes ranged from $N = 8$ up to $N = 128$. Metrics recorded include library import overhead (`import_time`, measured via `time.perf_counter`), model construction time (`build_time`), active solving time (`solve_time`), variable count, and clause/constraint count. Each benchmark configuration was executed over 3 repetitions, reporting median values. Commercial solver Gurobi operated under trial license limitations ($N \le 44$), while CPLEX operated under full license support allowing scaling up to $N = 128$.

4. Empirical Results and Performance Analysis

4.1. SAT Solvers Performance

Table 1 summarizes SAT benchmark results across diverse encodings and solvers (total time in seconds: import + build + solve).

Table 2 presents the model size (variable and clause counts) across board sizes N and encoding variants.

4.2. CP-SAT Performance

Table 3 outlines OR-Tools CP-SAT performance.

4.3. Open-Source MIP Performance

Table 4 summarizes CBC and SCIP performance.

Table 1
SAT Solvers Performance Metrics (Total Time in Seconds: Import + Build + Solve)

N	Encoding	CaDiCaL	Glucose3	MiniSat22	Kissat	MapleCM
8	binary	0.011	0.010	0.010	0.015	0.010
8	commander	0.010	0.010	0.010	0.013	0.010
8	pairwise	0.010	0.010	0.010	0.014	0.010
8	product	0.010	0.010	0.010	0.014	0.010
8	sequential	0.010	0.010	0.010	0.013	0.010
16	binary	0.017	0.095	0.013	0.030	0.019
16	commander	0.015	0.026	0.013	0.032	0.047
16	pairwise	0.014	0.013	0.012	0.033	0.015
16	product	0.017	0.077	0.013	0.030	0.031
16	sequential	0.022	0.024	0.012	0.034	0.020
32	binary	0.039	0.287	0.019	0.049	0.433
32	commander	0.074	1.471	0.099	0.188	0.421
32	pairwise	0.036	0.031	0.022	0.078	0.057
32	product	0.043	0.797	0.030	0.053	0.590
32	sequential	0.054	0.183	0.021	0.094	1.301
64	binary	0.320	1.549	0.098	0.901	0.639
64	commander	0.328	T/O	0.218	0.393	0.976
64	pairwise	0.186	0.224	0.110	0.651	0.725
64	product	0.963	T/O	0.265	0.412	1.121
64	sequential	1.208	T/O	0.165	0.091	9.828
128	binary	1.440	T/O	0.193	2.314	T/O
128	commander	0.408	T/O	T/O	1.458	T/O
128	pairwise	1.532	8.100	1.098	6.027	1.783
128	product	2.225	T/O	T/O	1.325	26.222
128	sequential	8.599	T/O	T/O	1.390	T/O

4.4. Commercial Gurobi and CPLEX Performance

Tables 5 and 6 present Gurobi and CPLEX results.

5. Timeout Analysis

Table 7 details the timeout counts encountered across solver paradigms and board sizes under the strict 30-second limit.

Table 2
SAT Model Size Metrics by Encoding Variant

N	Encoding	Variables	Clauses
8	binary	174	720
8	commander	146	528
8	pairwise	64	736
8	product	412	880
8	sequential	274	596
16	binary	572	3912
16	commander	682	2610
16	pairwise	256	6336
16	product	1188	3208
16	sequential	1186	2716
32	binary	1850	19768
32	commander	2956	11326
32	pairwise	1024	52608
32	product	3740	13036
32	sequential	4930	11564
64	binary	6136	95512
64	commander	11958	47096
64	pairwise	4096	428800
64	product	11064	46944
64	sequential	20098	47692
128	binary	21238	447704
128	commander	48836	192754
128	pairwise	16384	3462656
128	product	35598	182888
128	sequential	81154	193676

6. Visualizations

7. Discussion and Comprehensive Solver Comparison

The empirical results reveal distinct performance characteristics and limitations across all six solver groups (SAT, CP-SAT, open-source MIP, Gurobi, CPLEX MIP, and CPLEX CP) when scaling across the N range ($N = 8$ to 128):

- **Comparative Performance Across Six Solver Groups**: Across moderate to large scales ($N = 8$ to 128), commercial engines (CPLEX MIP, CPLEX CP, Gurobi) and high-performance solvers (CP-SAT, CaDiCaL) demonstrate exceptional scalability. At $N = 128$, recorded runtimes range from approximately 10.5 ms for CPLEX CP to 0.713 s for CP-SAT (noting that measurement methodologies differ across solvers, where reported `time_seconds` is the median over repetitions of each run's `import_time + build_time + solve_time` — with build and solve medians reported separately so they may not sum exactly — excluding parent-side process launch overhead). Crucially, `import_time` is library-dependent and constant with respect to N (for instance, OR-Tools CP-SAT incurs ≈ 0.3 s of Python module import overhead regardless of N), meaning cross-paradigm runtime comparisons at small N primarily reflect library import latency rather than algorithmic solving speed. To isolate solver efficiency from module import overhead, Figure 2 (Plot 2) evaluates *model time* (`build_time + solve_time`), with timeouts counted as 30 s, whereas many SAT combinations take tens of seconds or timeout.

- **FEASIBLE vs. OPTIMAL Status Semantics**: For pure satisfaction problems (such as N-Queens without an objective function), finding any valid non-attacking configuration successfully solves the problem. Solution status labels are solver-specific and API-dependent: CPLEX CP reports `FEASIBLE`,

Table 3
CP-SAT Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	allowed	ortools-cpsat	8	29	0.183983
8	boolean	ortools-cpsat	64	46	0.179115
8	integer	ortools-cpsat	23	3	0.178870
16	allowed	ortools-cpsat	16	121	0.200109
16	boolean	ortools-cpsat	256	94	0.183922
16	integer	ortools-cpsat	47	3	0.182438
32	allowed	ortools-cpsat	32	497	0.394736
32	boolean	ortools-cpsat	1024	190	0.198667
32	integer	ortools-cpsat	95	3	0.197633
64	allowed	ortools-cpsat	64	2017	3.161940
64	boolean	ortools-cpsat	4096	382	0.270635
64	integer	ortools-cpsat	191	3	0.269267
128	allowed	ortools-cpsat	-	-	T/O
128	boolean	ortools-cpsat	16384	766	0.731344
128	integer	ortools-cpsat	383	3	0.713260

Table 4
MIP (CBC/SCIP) Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	binary	cbc	64	46	0.038747
8	binary	scip	64	46	0.039792
8	integer	cbc	72	54	0.037998
8	integer	scip	72	54	0.040282
16	binary	cbc	256	94	0.055559
16	binary	scip	256	94	0.040704
16	integer	cbc	272	110	0.049471
16	integer	scip	272	110	0.041553
32	binary	cbc	1024	190	0.087822
32	binary	scip	1024	190	0.730311
32	integer	cbc	1056	222	0.085984
32	integer	scip	1056	222	0.700315
64	binary	cbc	4096	382	0.343513
64	binary	scip	4096	382	1.332685
64	integer	cbc	4160	446	0.431371
64	integer	scip	4160	446	3.625789

whereas CP-SAT reports `OPTIMAL` and MIP solvers may report `OPTIMAL` for satisfaction models. Solution validity in our framework is independently verified using the benchmark's verification module rather than relying solely on solver status strings.

- **CP Optimizer Process Startup Overhead**: CP Optimizer relies on an external binary executable (`cpoptimizer`). Spawning this subprocess incurs inter-process communication and process startup overhead; actual total measured runtimes for CPLEX CP increase from approximately 0.3 ms at $N = 8$ to 10.5 ms at $N = 128$. Note that the `solve_time` of CPLEX CP comes from `get_solve_time()`, which in CSV records values of 0.00 and 0.01 (indicating a resolution of 10 ms); thus, the 10.5 ms recorded at $N = 128$ appears to be predominantly 10 ms rounding plus 0.5 ms model build time, and should not be directly compared on equal footing with CP-SAT's 0.713 s wall-clock measurement.

Table 5
Gurobi Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	binary	gurobi	64	42	0.000948
16	binary	gurobi	256	90	0.001468
32	binary	gurobi	1024	186	0.011580
44	binary	gurobi	1936	258	0.005287

Table 6
CPLEX Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Total Time (s)
8	binary	cplex	64	42	0.010834
8	integer	cplex-cp	8	3	0.000264
16	binary	cplex	256	90	0.012104
16	integer	cplex-cp	16	3	0.000287
32	binary	cplex	1024	186	0.016284
32	integer	cplex-cp	32	3	0.000320
64	binary	cplex	4096	378	0.032064
64	integer	cplex-cp	64	3	0.000400
128	binary	cplex	16384	762	0.092598
128	integer	cplex-cp	128	3	0.010534

Table 7
Timeout Analysis Across Solvers and Board Sizes

Paradigm	Solver	Board Size (N)	Timeout Count
CP-SAT	ortools-cpsat	128	1
SAT	glucose3	64	3
SAT	glucose3	128	4
SAT	maplecm	128	3
SAT	minisat22	128	3

- **Gurobi Trial License Limitations**: Gurobi's trial license restricts model dimensions for larger instances, capping trials at $N \leq 44$. For $N > 44$, full commercial licenses or alternative solvers (CPLEX, CP-SAT, CaDiCaL) are required.
- **SAT Scaling and Encodings**: CaDiCaL and Kissat handle large clause databases efficiently up to $N = 128$ (with maximum recorded runtimes of approximately 8.60 s for CaDiCaL and 6.03 s for Kissat), whereas MapleCM reaches a maximum recorded runtime of approximately 26.22 seconds under product encoding at $N = 128$, and Glucose3, MiniSat22, and MapleCM encounter frequent timeouts (10/25 cases timed out at $N = 128$, specifically Glucose3: 4, MiniSat22: 3, MapleCM: 3). Compact cardinality encodings such as Commander (47,096 clauses at $N = 64$) and Sequential Counter (47,692 clauses at $N = 64$) dramatically reduce clause footprints compared to naive Pairwise encodings.
- **CP-SAT Allowed Assignment Timeout at** $N = 128$: At $N = 128$, the CP-SAT allowed assignment variant timed out within 30 s; the underlying cause has not been isolated.

8. Conclusion

This comprehensive benchmark demonstrates that while modern SAT, CP-SAT, and commercial CP/MIP engines solve moderate instances efficiently, performance varies significantly at $N = 128$. Performance is heavily dictated by solver sensitivity, global constraint propagation, and encoding efficiency: CaDiCaL, Kissat, CPLEX CP, and CP-SAT handle large scales effectively, whereas clausal blow-up and solver heuristics cause frequent timeouts for older SAT engines.

9. References

References

[1] A. Biere, "CaDiCaL at SAT Race 2019," in *Proceedings of SAT Race 2019* (Pragmatics of SAT, Vol. B), pp. 1–2, Department of Computer Science, University of Helsinki, 2019. Available: http://www.satcompetition.org/2019/proceedings.pdf

[2] C. Sinz, "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints," In *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827–831, 2005.

[3] W. Klieber and G. Kwon, "Efficient CNF Encoding for Selecting 1 from n Objects," In *Constraints in Formal Verification (CFV)*, 2007.

[4] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," In *International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 428–437, 2018.

[5] J. Chen, "A New SAT Encoding for the At-Most-One Constraint," In *International Workshop on Modelling and Reformulation (ModRef)*, 2010.

[6] L. Perron and V. Furnon, "OR-Tools," Google, 2021. Available: https://developers.google.com/optimization.

[7] A. Biere, M. Heule, H. van Maaren, and T. Walsh, *Handbook of Satisfiability*, 2nd ed. IOS Press, 2021.

10. Data Availability

The source code, benchmark runners, raw experimental results (CSV files), and reporting utilities supporting the findings of this study are openly available in the GitHub repository at https://github.com/datnk0116/Nqueen_KHMT.

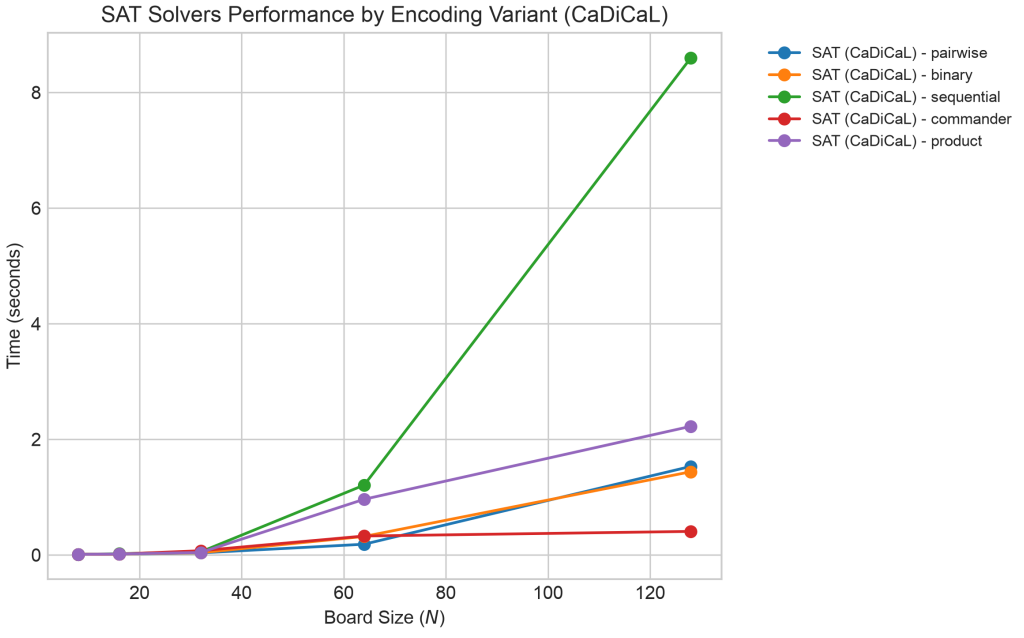


Figure 1: SAT (CaDiCaL) total time scaling across board sizes N.

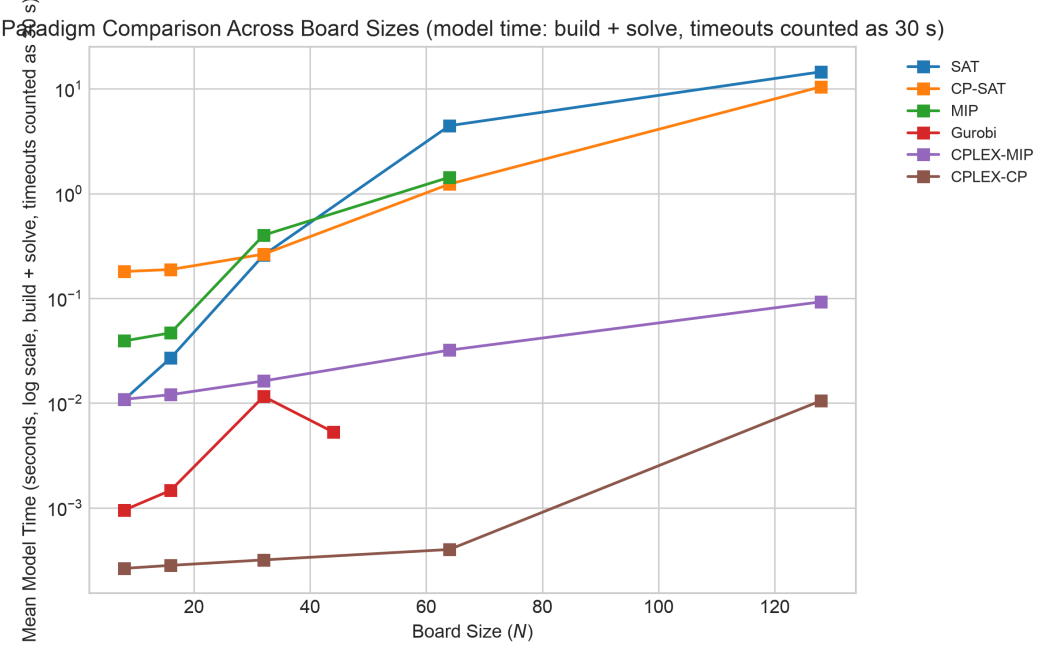


Figure 2: Comparison of mean model time across all solver paradigms (build + solve; log scale; timeouts counted as 30 s, so means are lower bounds).

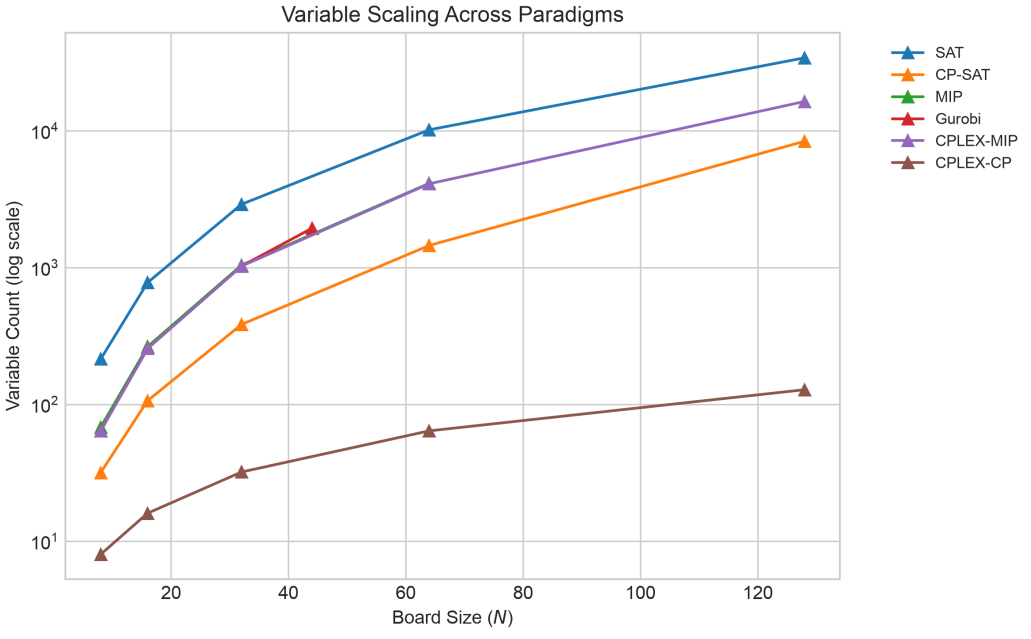


Figure 3: Variable count scaling across different solving paradigms and solvers.

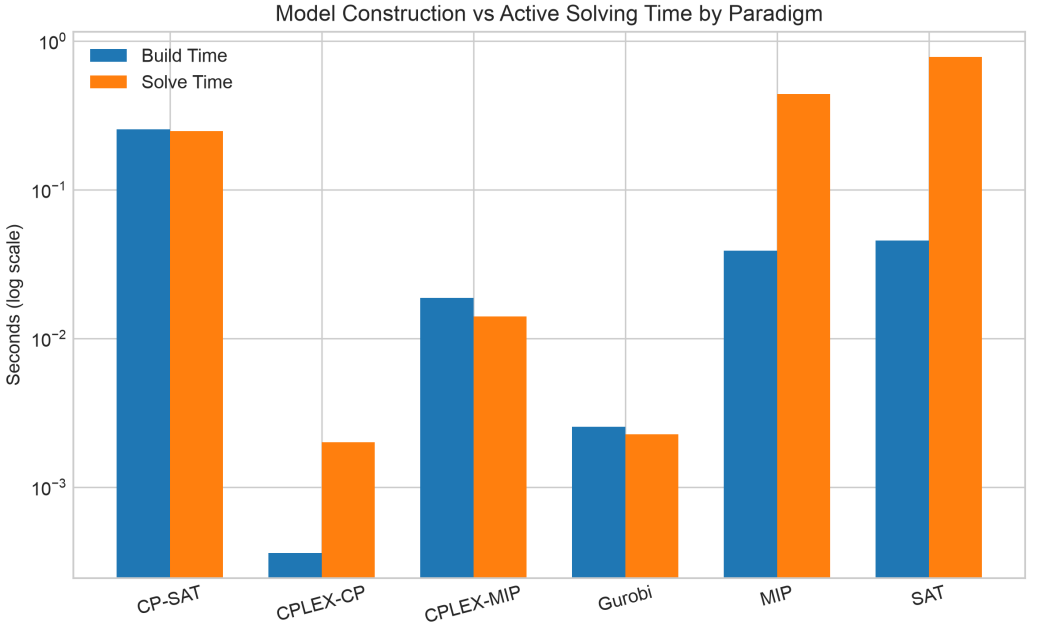


Figure 4: Model construction overhead versus active solving time by paradigm.